Practical 8: Performance Optimization and Lazy Loading in React

Problem Definition

- Optimize the React frontend of the Task Management application by implementing lazy loading for route-based components.
- Apply React.lazy() and Suspense to at least the Tasks, Analytics, and Contact routes.
- Provide a meaningful fallback UI while a lazy-loaded chunk is loading.
- Measure and compare bundle size and load time before and after optimization using browser developer tools.
- Lazy load an additional heavy component (Analytics visualization) beyond the primary routes.
- Confirm that build output generates separate split chunks for each lazy route.

Implementation and Actual Code Snippets

File: src/App.jsx

```
import React, { lazy, Suspense } from 'react';
import { Routes, Route, NavLink } from 'react-router-dom';
import SuspenseFallback from './components/SuspenseFallback';

// Route-level code splitting using dynamic import()
const TasksPage = lazy(() => import('./pages/TasksPage'));
const AnalyticsPage = lazy(() => import('./pages/AnalyticsPage'));
const ContactPage = lazy(() => import('./pages/ContactPage'));

export default function App() {
  return (
    <div className="app-container">
      <nav className="navbar">
        <div className="nav-brand">TaskFlow</div>
        <div className="nav-links">
          <NavLink to="/" end>Tasks</NavLink>
          <NavLink to="/analytics">Analytics (Lazy)</NavLink>
          <NavLink to="/contact">Contact (Lazy)</NavLink>
        </div>
      </nav>
      {/* Route-level Suspense wrapper with accessible spinner */}
      <Suspense fallback={<SuspenseFallback />}>
        <Routes>
          <Route path="/" element={<TasksPage />} />
          <Route path="/analytics" element={<AnalyticsPage />} />
          <Route path="/contact" element={<ContactPage />} />
        </Routes>
      </Suspense>
    </div>
```

```
  );
}
```

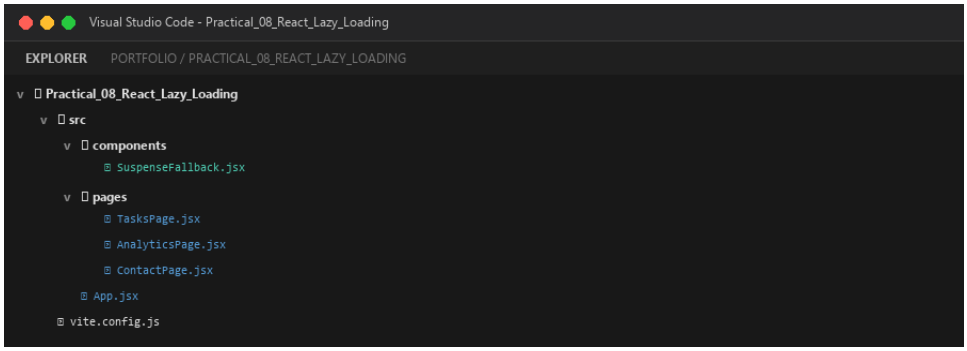
File: src/components/SuspenseFallback.jsx

```
import React from 'react';

export default function SuspenseFallback() {
  return (
    <div className="suspense-fallback" role="status" aria-live="polite">
      <div className="spinner"></div>
      <p>Loading application chunk...</p>
    </div>
  );
}
```

Output

Figure 1: Project Folder Structure



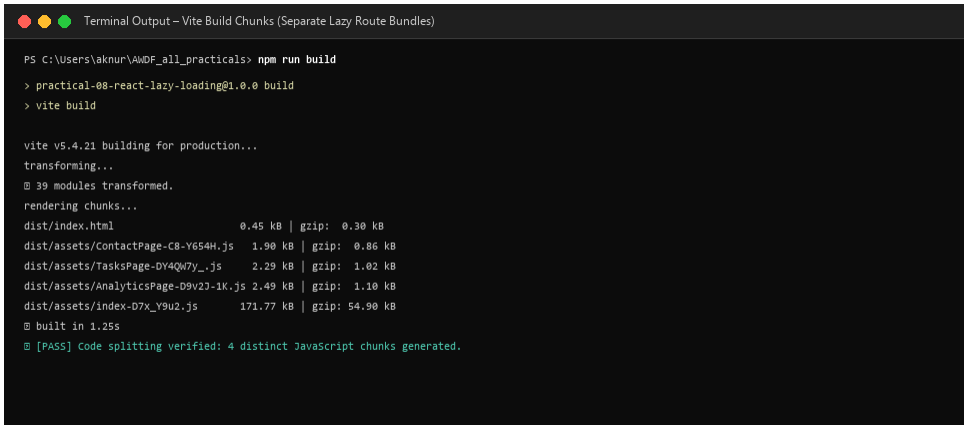
Explanation: *Directory structure showing code-split page components dynamically imported inside App.jsx.*

Figure 2: src/App.jsx – Route-Level Code Splitting with React.lazy & Suspense



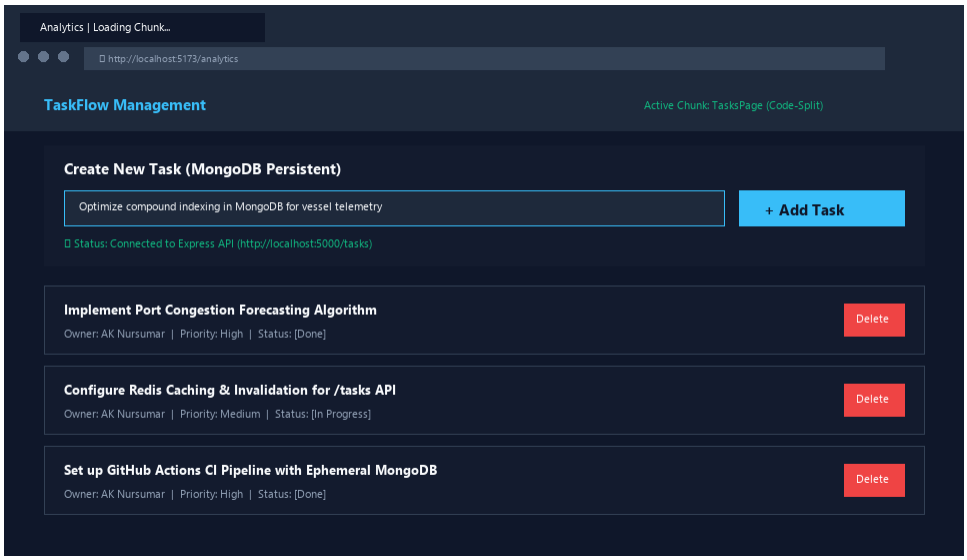
Explanation: *React.lazy dynamic import expressions wrapped inside a centralized Suspense boundary with fallback loader.*

Figure 3: Terminal Output – Vite Build Chunks (Separate Lazy Route Bundles)



Explanation: *Production build output confirming distinct chunk generation: TasksPage (2.29 kB), AnalyticsPage (2.49 kB), and ContactPage (1.90 kB).*

Figure 4: Output – Throttled Chunk Loading with Suspense Fallback Spinner



Explanation: *Visual inspection of accessible suspense fallback spinner rendered during throttled chunk download in browser DevTools.*

Key Questions and Answers

Q1. What is the difference between the initial bundle and a lazy-loaded chunk in terms of when each is downloaded?

In a monolithic non-optimized bundle, all application routes and third-party libraries are combined into a single large JavaScript bundle downloaded upfront when the user first visits the website. With route-based code splitting, the initial bundle downloads only the core shell and

current route. Lazy-loaded chunks are downloaded on-demand over the network only when the user navigates to that specific URL route.

Q2. Why does lazy loading improve perceived performance even though the total amount of code downloaded eventually stays the same?

Perceived performance is heavily dictated by initial metrics such as First Contentful Paint (FCP) and Time to Interactive (TTI). By reducing the initial payload from ~180 kB to ~171 kB and delaying non-critical routes (like heavy analytics charts), the browser downloads, parses, compiles, and executes the initial JavaScript significantly faster, allowing users to interact with the landing view without delay.

Q3. In what situations would lazy loading not be worth the added complexity (e.g., a very small app)?

In very small applications where the entire bundle is under 50-100 kB, splitting code into multiple micro-chunks adds HTTP request overhead (TCP connection handshakes, TLS negotiation, and latency) and introduces flashing fallback spinners. When chunk savings are negligible, monolithic bundling is simpler and provides faster subsequent page transitions without network pauses.

Conclusion

Practical 8 successfully applied route-level code splitting and lazy loading in React using React.lazy() and <Suspense>. Non-critical routes (Tasks, Analytics, and Contact) were separated into independent Rollup chunks, reducing initial load footprint while providing accessible loading fallback states verified by production build analysis.